

KẾ HOẠCH TÍNH NĂNG MỚI — GHI CHÚ TRONG NGÀY

Đánh giá SQLMesh/Debezium làm Related Work, và thiết kế 4 tính năng mở rộng (Schema Editor, Merge, Sandbox, Task Monitor). Deadline nghiệm thu: tháng 10 — còn nhiều thời gian, dự kiến 1-2 tuần tập trung cho phần này.

1. SQLMesh — KHÔNG đưa vào đề tài

Kết luận: SQLMesh quản lý transformation logic (câu SQL biến đổi dữ liệu), không phải schema structure (cấu trúc bảng) — khác phạm vi với đề tài. "Breaking/non-breaking" của SQLMesh dựa trên việc giá trị dữ liệu tính ra có đổi hay không, không liên quan gì đến việc cột có đổi kiểu/bị xóa hay không.

Quyết định: không đưa SQLMesh vào Chương III (Related Work) vì lạc phạm vi, dễ bị hỏi vặn "sao tự nhiên có dbt-like tool ở đây". Chỉ có 1 phần rất nhỏ (column-level lineage) có thể tận dụng SAU NÀY nếu làm tính năng Impact Analysis — không phải bây giờ.

2. Debezium — dùng MỘT PHẦN, không dùng nguyên khối

Debezium chỉ đóng vai cảm biến (phát hiện CÓ DDL xảy ra qua binlog/WAL, ghi vào Kafka topic riêng) — không phân loại breaking/non-breaking, không Approval Workflow, không tự sinh DDL đồng bộ, không lưu version để tra cứu. Toàn bộ phần "bộ não" (compare_schemas, Evolution Policies, SchemaRegistry, Approval) vẫn giữ nguyên của hệ thống đã xây.

Quyết định: chỉ dùng Debezium làm TRIGGER — nghe Kafka topic schema-changes, khi có message thì gọi lại run_evolution_check() ngay lập tức thay vì đợi hết chu kỳ polling. Giữ nguyên polling định kỳ song song làm lưới an toàn. Đây là hiện thực hoá "Hướng nâng cao #1" (Polling → Event-driven) đã note trước đó. Việc này để làm SAU (chưa code ngay), ưu tiên 4 tính năng mới trước.

3. Bốn tính năng mới — thiết kế trong ngày

3.1. Schema Editor low-code (cho non-IT)

Mục tiêu: người dùng không biết SQL vẫn sửa được cấu trúc bảng. Streamlit không hỗ trợ kéo-thả thật (cần custom component React/JS riêng, tốn nhiều thời gian). Ông quyết định giao phần frontend kéo-thả cho công cụ/người khác (antigravity) làm riêng.

Quyết định: Claude tập trung phần BACKEND — logic diff/generate_ddl/approve đã có sẵn, chỉ cần frontend (bất kỳ ai làm) gọi đúng các hàm này. Backend không cần đổi gì thêm cho tính năng này, chỉ cần expose đúng API/hàm cho frontend gọi vào.

3.2. Merge vào DB đích/nguồn sau khi sửa

Đã có sẵn — chính là luồng approve_change_and_sync() đã code và test xong ở phiên trước (tính lại diff đầy đủ từ draft, sinh DDL, apply lên target, cập nhật Registry). Tính năng Schema Editor (3.1) chỉ cần tái sử dụng đúng luồng này, không cần viết logic merge mới.

3.3. Sandbox nhẹ — test DDL an toàn trước khi áp dụng thật

Không cần Docker-in-Docker hay clone cả database. Chỉ cần clone CẤU TRÚC (không data) của đúng 1 bảng đang sửa thành 1 bảng nháp ngay trong cùng DB, chạy thử DDL lên bảng nháp đó, bắt lỗi cú pháp/logic thật (từ chính engine thật), rồi LUÔN xóa bảng nháp — dù thành công hay thất bại.

1. Sinh DDL (generate_ddl co san)
2. Tao bang nhap:

```
CREATE TABLE sandbox_customers_a8f3 (LIKE customers) -- Postgres
CREATE TABLE sandbox_customers_a8f3 LIKE customers -- MySQL
CREATE TABLE sandbox_customers_a8f3 AS customers -- ClickHouse
```
3. Chạy DDL do len bang NHAP (khong phai bang that)
4. Bat loi neu co -> bao 'Test that bai: <loi cu the>'
5. LUON xoa bang nhap (finally block, du thanh cong hay that bai)
6. Chi khi test pass moi cho phep bam 'Ap dung len bang that'

Cần thêm 2 method mới vào BaseEngine (không phá interface cũ, không bắt buộc mọi driver đều dùng): clone_table_structure(table_name, sandbox_name) và drop_table(table_name). Mongo không cần implement thật (schema-less, DDL vốn no-op sẵn — sandbox cho Mongo luôn "pass" trơn). Thêm 1 hàm điều phối test_changes_in_sandbox() trong core.py, dùng dataclasses.replace() để đổi table_name trong SchemaChange sang tên sandbox, không sửa trực tiếp changes gốc.

Quyết định: làm bản NHẸ — chỉ test cấu trúc rỗng, KHÔNG copy data mẫu vào bảng nháp (đơn giản hơn, đủ để bắt lỗi cú pháp/logic DDL — loại lỗi nguy hiểm nhất đã từng gặp nhiều lần trong quá trình code). Nối vào UI: thêm 1 nút "Test trước khi áp dụng" cạnh nút Approve, chỉ khi test pass mới cho bấm Approve thật.

3.4. Task/Flow Monitor — danh sách theo dõi

Rủi ro thấp nhất trong 4 cái — mở rộng tự nhiên từ app.py (Streamlit) đã có. Thêm 1 trang tổng quan: liệt kê tất cả pair/bảng đang theo dõi, trạng thái mới nhất, lần quét gần nhất, kết quả gần nhất. Hầu hết dữ liệu ĐÃ CÓ SẴN trong Registry (is_frozen, version, updated_at) — chỉ cần code hiển thị tổng hợp thay vì hiện từng bảng riêng lẻ như hiện tại.

4. Thứ tự ưu tiên đề xuất

#	Việc cần làm	Ghi chú
1	Sandbox backend (3.3)	Claude code — nền tảng để Schema Editor (frontend riêng) gọi vào an toàn
2	Task/Flow Monitor (3.4)	Claude code — rủi ro thấp, mở rộng app.py có sẵn
3	Schema Editor frontend (3.1)	Antigravity/người khác code phần kéo-thả, gọi vào backend đã có
4	Debezium event-driven consumer	Để sau — mở rộng kiến trúc, không phải tính năng người dùng trực tiếp

5. Kết luận

Không đưa SQLMesh vào báo cáo (lạc phạm vi). Debezium chỉ mượn phần capture DDL làm trigger, giữ nguyên toàn bộ logic đã xây. Bốn tính năng mới: Schema Editor (frontend giao cho antigravity, backend tái dùng luồng có sẵn), Merge (đã xong, tái dùng approve_change_and_sync), Sandbox (làm bản nhẹ — clone cấu trúc rỗng, test DDL, dọn dẹp), Task Monitor (mở rộng app.py, rủi ro thấp). Deadline tháng 10, không áp lực thời gian — làm từng phần, test kỹ như cách đã làm với core.py trước đó, tránh lặp lại các lỗi phải sửa đi sửa lại nhiều lần.